

Module

Tim Schmitz
05.07.2023

Inhalt

Definition	3
Decorator	3
Modularten	4
Hauptmodul	5
Feature-Module	5
Routing-Module	5
Service-Module	5
Widget-Module	6

Angular-Module

Angular-Module haben die Aufgabe, eine Webanwendung in verschiedene Verantwortungsbereiche und Funktionsblöcke zu gliedern. Alle Bausteine, die logisch zu diesem Bereich gehören, werden mit dem entsprechenden Angular-Modul registriert. Die so registrierten Bausteine gehören damit zu dem Angular-Modul. Die Idee, die ECMAScript hier verfolgt, ist, dass TypeScript-Konstrukte wie Klassen und Funktionen gekapselt werden und nur der Zugriff auf exportierte Konstrukte gestattet wird. Das heißt, dass ein anderes Angular-Modul nur auf die Bausteine (Komponenten, Pipes oder Direktiven) zugreifen darf, die von dem eigenen Angular-Modul exportiert werden.

Definition

Ein Angular-Modul oder kurz NgModule ist eine in TypeScript geschriebene Klasse, die ein spezielles Konstrukt nutzt, den NgModule-Decorator.

Der Decorator, der durch das Adresszeichen @ gekennzeichnet ist, wird für Klassen und ihre Eigenschaften, Methoden und Parameter genutzt, die zur Laufzeit aufgerufen werden, um die Funktionalität der Klasse zu erweitern. Decoratoren konfigurieren damit Angular-Bausteine und definieren Metadaten.

Jedes Angular-Modul hat sein eigenes, dediziertes Verzeichnis. Der Name des Verzeichnisses beschreibt die Funktionalität des Moduls. Das Verzeichnis enthält die Datei für das Angular-Modul, die über den Dateinamen gekennzeichnet ist, sowie alle Angular-Bausteine, die zu diesem Angular-Modul gehören.

Das folgende Code-Listing zeigt den Inhalt der Datei app.module.ts, die von der Angular-Entwicklungsumgebung standardmäßig generiert wurde und als Hauptmodul für das Projekt fungiert.

```
import { BrowserModule } from '@angular/platform-browser';
import { NgModule } from '@angular/core';
import { AppComponent } from './app.component';

@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Zunächst werden drei Konstrukte importiert. Zwei Konstrukte kommen von der Angular-Laufzeitumgebung selbst, wie man an der Kodifizierung mit dem Adresszeichen sieht, und ein Konstrukt ist die zu dem Modul dazugehörige Komponente. Die Module der Angular-Laufzeitumgebung müssen daher in der package.json-Konfigurationsdatei aufgeführt und per npm installiert werden.

Anschließend werden einige Eigenschaften des Hauptmoduls dekoriert. Zum Schluß wird das Hauptmodul als Klasse exportiert. Zwischen diesen beiden Bestandteilen darf kein eigener Code eingefügt werden, da es sonst zur Laufzeit eine Exception wirft.

Das BrowserModule muss in jeder Angular-Anwendung importiert werden, damit die Anwendung mit dem Browser interagiert. Es darf allerdings auch nur einmal importiert werden.

Decorator

Der NgModule-Decorator hat fünf optionale Eigenschaften. Die Werte dieser Eigenschaften sind alle Arrays. Wenn eine Eigenschaft keinen Wert besitzt, dann muss dieser auch nicht angegeben werden. Wie im Listing beschrieben, so hat der Decorator etwa des Angular-Hauptmoduls folgendes Aussehen:

```
@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule],
  providers: [],
  bootstrap: [AppComponent]
})
```

declarations

Hier deklariert man die Komponenten, Direktiven und Pipes, die zu diesem Angular-Modul gehören. Im Beispiel ist nur die Hauptkomponente Teil des Angular-Moduls.

providers

Hier registriert man Providers für das Angular-Modul. Providers braucht man, wenn man mit Services arbeitet. Providers, die hier registriert sind, stehen der gesamten Anwendung zur Verfügung. In dem obigen Beispiel existiert noch kein Provider für das Angular-Hauptmodul.

exports

Hier exportiert man die Komponenten, Direktiven und Pipes aus dem eigenen Angular-Modul, die für andere Angular-Modulen zur Verfügung stehen sollen. Dieser Array kann aber auch weitere Angular-Module exportieren. In diesem Fall hat jedes Angular-Modul, das dieses Angular-Modul importiert den Zugriff auf die Exporte dieses Moduls sowie jeder anderen exportierten Module. Im Beispiel werden keine anderen Bausteine von dem Angular- Hauptmodul exportiert.

imports

Dieses Array beinhaltet die Angular-Module, von denen das aktuelle Angular-Modul abhängt. Das können externe Angular-Module aus der Angular-Laufzeitumgebung wie das BrowserModule sein oder aber auch eigene Angular-Module. In diesem Fall stehen dem importierenden Angular-Modul dann alle Bausteine zur Verfügung, die von den importierten Angular-Modulen exportiert werden. Wie bereits oben beschrieben stehen die Providers von importierten Angular-Modulen der gesamten Anwendung zur Verfügung. Das Beispiel zeigt den Import des wesentlichen BrowserModuls für das Angular-Hauptmodul.

bootstrap

Die optionale Eigenschaft bootstrap definiert die Hauptkomponente der Anwendung, die durch einen Klassennamen mit dem Bestandteil Component gekennzeichnet ist. Die Hauptkomponente der Anwendung wird nur im Angular-Hauptmodul benutzt. Die Hauptkomponente ist die Wurzel des Komponentenbaums.

Modularten

Man kann Angular-Module unterscheiden nach den Arten dieser Module. Die Arten unterscheiden sich in der Rolle, die diese Angular-Module innerhalb einer Anwendung spielen. Technisch gesehen sind alle Modularten Klassen, die mit dem NgModule-Decorator dekoriert werden. Allerdings hilft es bisweilen, verschiedene Arten von Angular-Modulen zu unterscheiden, um innerhalb der Anwendung eine Strukturierung zu erzielen. Man kann folgende Modularten unterscheiden:

- Hauptmodul
- Feature-Module
- Routing-Module
- Service-Module
- Widget-Module

Hauptmodul

Das Angular-Hauptmodul spielt in einer Angular-Anwendung eine spezielle und wesentliche Rolle. Die hauptsächliche TypeScript-Datei der Anwendung `main.ts` importiert das Hauptmodul `app.module.ts` als Klasse `AppModule` und übergibt es der Initialisierungsfunktion:

```
import { AppModule } from './app/app.module';
...
platformBrowserDynamic().bootstrapModule(AppModule)
  .catch(err => console.error(err));
```

Falls es in der Anwendung noch weitere Feature-Module gibt, so werden sie hier ebenfalls importiert.

Feature-Module

Die Features einer Anwendung werden in kleinere Funktionsblöcke aufgeteilt. Ein Feature-Modul repräsentiert damit einen Funktionsblock (Feature) einer Anwendung. Es gibt in der regel ein Feature-Modul, das die Hauptfunktion der Anwendung enthält. Feature-Module können in ihrerseits in Untermodule aufgesplittet werden.

Beispiel: Die Beispiel-Anwendung soll zwei Feature-Module mit folgenden Funktionalitäten beinhalten:

- `ContactsListsModule` verwaltet Kontaktlisten
- `ContactsModule` verwaltet Kontakte

In dem Feature-Modul müssen wie immer folgende Objekte deklariert werden, um das Feature zu implementieren:

- Komponenten
- Pipes
- Direktiven

Feature-Module importieren oft auch weitere Angular-Module. Zum Beispiel enthält das Kontaktlisten-Modul ein Formular. Damit dessen Funktionalität dem Feature-Module zur Verfügung steht, muss es das `ReactiveFormsModule` von Angular importieren.

Umgekehrt sollte man ein Feature-Module nur in einem Angular-Modul importiert werden. In der Regel importiert man ein Feature-Modul im Hauptmodul der Anwendung. Untermodule werden vom entsprechenden Feature-Modul importiert.

Routing-Module

Routing-Module enthalten die Konfiguration für den Angular-Router. Der Angular-Router ist eine Kernfunktionalität der Angular-Plattform und erlaubt die Entwicklung von Single Page-Applikationen mit vielen Sichten und die Navigation zwischen diesen Sichten. Es gibt ein Routing-Modul für die gesamte Anwendung. Für jedes Feature-Modul gibt es eigene Routing-Module.

In der Beispielanwendung existieren die folgenden Routing-Module:

- `AppRoutingModule` (für die gesamte Anwendung)
- `ContactsListRoutingModule` (für das Feature `ContactsListsModule`)
- `ContactsRoutingModule` (für das Feature `ContactsModule`)

Service-Module

Service-Module beinhalten Providers und stellen diese der gesamten Anwendung zur Verfügung. Zum Beispiel kann in solch einem Modul ein Service mit Hilfsfunktionen definiert werden. Ansonsten besitzen Service-Module keine Komponenten, Direktiven oder Pipes. Sie sollten nur in das Hauptmodul der Anwendung importiert werden. Beispielsweise ist das HttpClientModule von Angular ein Service-Modul.

Widget-Module

Widget-Module definieren User Interface-Bausteine, die wiederverwendbar sind wie zum Beispiel Komponenten. Ein Widget ist eine Kombination aus "Window" (Fenster) und "Gadget" (Vorrichtung). Damit diese UI-Bausteine in anderen Angular-Modulen genutzt werden können, müssen sie im declarations-Array deklariert werden und im exports-Array exportiert werden. Die anderen Angular-Module müssen dann diese Widget-Module importieren, um die UI-Bausteine nutzen zu können. Beispielsweise ist das CommonModule von Angular selbst ein Widget-Module.